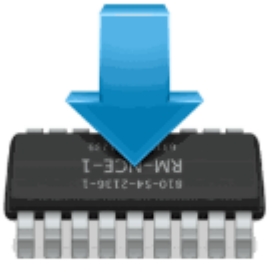
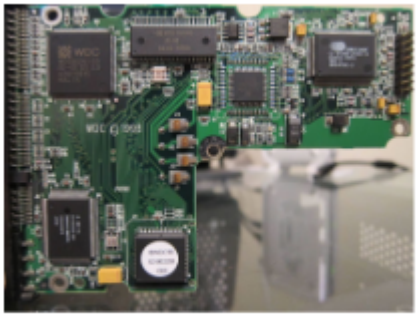
I fundamentals di un Sistema Operativo

Firmware

Il **firmware** è un tipo di software integrato all'interno di componenti hardware del dispositivo;
come ad esempio: processori, schede grafiche, schede audio, schede di rete e dispositivi periferici (come stampanti e monitor).
La sua **funzione principale** è:
quella di inizializzare il componente e facilitare la sua comunicazione con gli altri componenti all'interno del dispositivo.
In sostanza, il firmware fornisce al componente un insieme di interfacce di comunicazione e protocolli, permettendo ai vari componenti di **"parlare" tra di loro**, utilizzando un linguaggio comune (che comprende sintassi e semantica).





Firmware.png

Il firmware è un tipo di software **proprietario** e il termine deriva da **firm** e **ware** indicando che non è facilmente modificabile dall'utente:

- **funge da ponte tra hardware e software, consentendo così all'hardware di comunicare e funzionare secondo le istruzioni software** .
  Viene memorizzato in **memorie non volatili** (ROM), in modo che i dati non vengano persi quando si spegne il dispositivo.

Un esempio noto di firmware è il **BIOS** (Basic Input/Output System), che è un tipo di firmware più vecchio e **fondamentale per avviare il computer**. Il BIOS risiede sulla **scheda madre** e gestisce il processo iniziale di avvio del computer, inclusa l'inizializzazione dell'hardware e il caricamento del sistema operativo.

Anche dispositivi diversi, come **PC desktop, laptop, computer di bordo di auto o aeroplani**, contengono componenti simili, come le **schede madri**, che eseguono funzioni analoghe.

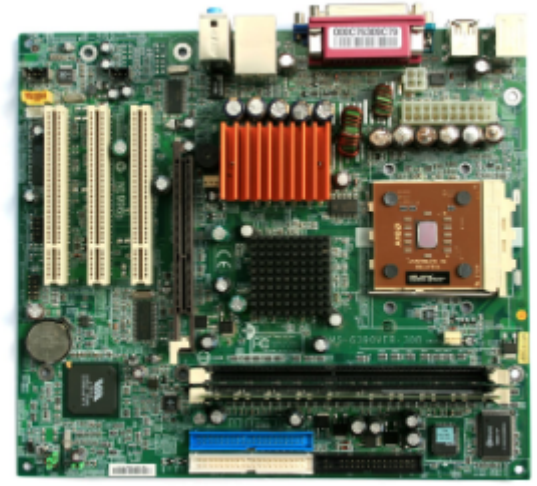
ROM (memoria non volatile)

I tipi più comuni di una memoria non volatile per archiviare il firmware sono:

- **ROM (memoria non volatile):**
  memoria non volatili la più famosa è la ROM:
  una volta scritta non può più essere modificata, adesso si utilizzano le evoluzioni
- **EPROM:**
  è una memoria che può essere cancellata e modificata utilizzando l'UV
- **EEPROM:**
  è come la EPROM ma anzichè usare l'UV utilizza l'elettricità.

Questi tipi di memoria sono integrati nel dispositivo, garantendo che il firmware sia facilmente accessibile per inizializzare e controllare i componenti del dispositivo quando viene acceso.
In tutte queste ROM la scrittura avviene bit per bit
LE flash memory sono un tipo di EEPROM ma hanno una capienza più grande utilizzate oggi per la memorizzazione di firmware perché possono essere memorizzate a blocchi quindi sono più veloci ed efficiente di fatto sono utilizzate anche nei SSD.

Motherboard



Motherboard.png

La **scheda madre** è una delle componenti più importanti all'interno di un dispositivo elettronico (non solo del computer).
È una **scheda a circuito stampato** (PCB) che ospita e consente la comunicazione tra i componenti elettronici fondamentali del dispositivo.
In sostanza funziona come un hub centrale, unendo tutte le parti del computer e permettendo loro di comunicare e lavorare insieme.

Sulla scheda madre si trovano diversi elementi cruciali:

- **Firmware**:
  Come il BIOS o l'UEFI, che forniscono le istruzioni per l'avvio del sistema e l'inizializzazione dell'hardware.
- **Slot per la memoria**:
  Qui vengono installati i moduli di RAM, che consentono al computer di immagazzinare e accedere rapidamente ai dati.
- **Slot di espansione**:
  Permettono l'aggiunta di carte aggiuntive, come schede grafiche, audio, di rete e altre periferiche.
- **Socket CPU**:
  Un connettore che ospita il processore centrale (CPU).
- **Connettori di memorizzazione**:
  Per dispositivi di archiviazione come dischi rigidi o SSD.
- **Connettori di alimentazione**:
  Permettono di fornire corrente alla scheda madre e ai suoi componenti tramite l'unità di alimentazione (PSU).
- **Connettori periferici**:
  Inclusi quelli per dispositivi USB, prese audio, connessioni di rete, e display (HDMI, DisplayPort).

In sintesi, la **scheda madre** è la spina dorsale di un computer, integrando tutti i componenti critici e garantendo che possano comunicare e funzionare insieme in modo efficace.

Bios (Basic Input/Output System)

Il **BIOS** è:

- **un insieme di routine software che forniscono la struttura di base che permette al sistema operativo di interfacciarsi con l'hardware del**

dispositivo.
È memorizzato in un chip di memoria non volatile:

- **ciò significa che mantiene i dati anche senza alimentazione**.
  Oggi, il BIOS è tipicamente memorizzato in memoria **FLASH** o **EEPROM** saldata direttamente sulla scheda madre, e può essere riscritto tramite una procedura di aggiornamento appropriata.

Processo di Bootstrap

Quando un dispositivo viene avviato, prima che il sistema operativo venga caricato, il BIOS esegue diverse operazioni fondamentali:

- **Schermo di avvio**: Mostra una schermata che consente all'utente di configurare e gestire alcune impostazioni di base del BIOS.
- **Avvio della routine di boot**: Il BIOS avvia la routine di avvio per caricare il sistema operativo. Il processo di avvio è incrementale, caricando le parti essenziali del sistema operativo dal settore di avvio per portare il sistema a uno stato operativo stabile.
- **Gestione dell'hardware**: Il BIOS esegue vari test, tra cui il **POST** (Power On Self Test), per verificare il corretto funzionamento dei vari componenti, come la scheda madre, il processore, la memoria, e altri.
- **Configurazione delle impostazioni hardware**: Se non vengono rilevati errori gravi durante la fase POST, il BIOS configura le impostazioni hardware, preparando il sistema affinché il sistema operativo possa prenderne il controllo.

Localizzazione dell'MBR (Master Boot Record)

Dopo aver eseguito le routine di base, la prima istruzione del BIOS è localizzare il **'punto di avvio'** del disco rigido (o di un altro media di archiviazione di massa) dove è memorizzato il sistema operativo. In questo punto si trova il **Master Boot Record** (MBR).

- **Controllo del sistema operativo:**
  - Il BIOS legge il contenuto dell'MBR e trasferisce il controllo al sistema operativo. Da questo momento in poi, la gestione del dispositivo è completamente affidata al sistema operativo.

In sostanza l'MBR contiene informazioni sulla posizione del **BOOT SECTOR** sul disco rigido (o altro media di archiviazione) dove è memorizzato il sistema

operativo.
- Il **BOOT SECTOR** è il punto di ingresso per l'avvio del sistema operativo.

Se la tabella **MBR** è danneggiata, il disco potrebbe non avviarsi correttamente, impedendo al computer di accendersi.

UEFI

UEFI è una moderna interfaccia firmware progettata per inizializzare i componenti hardware e caricare il sistema operativo all'avvio di un dispositivo.
Si tratta di un processo simile al BIOS tradizionale, **ma con diversi vantaggi**:

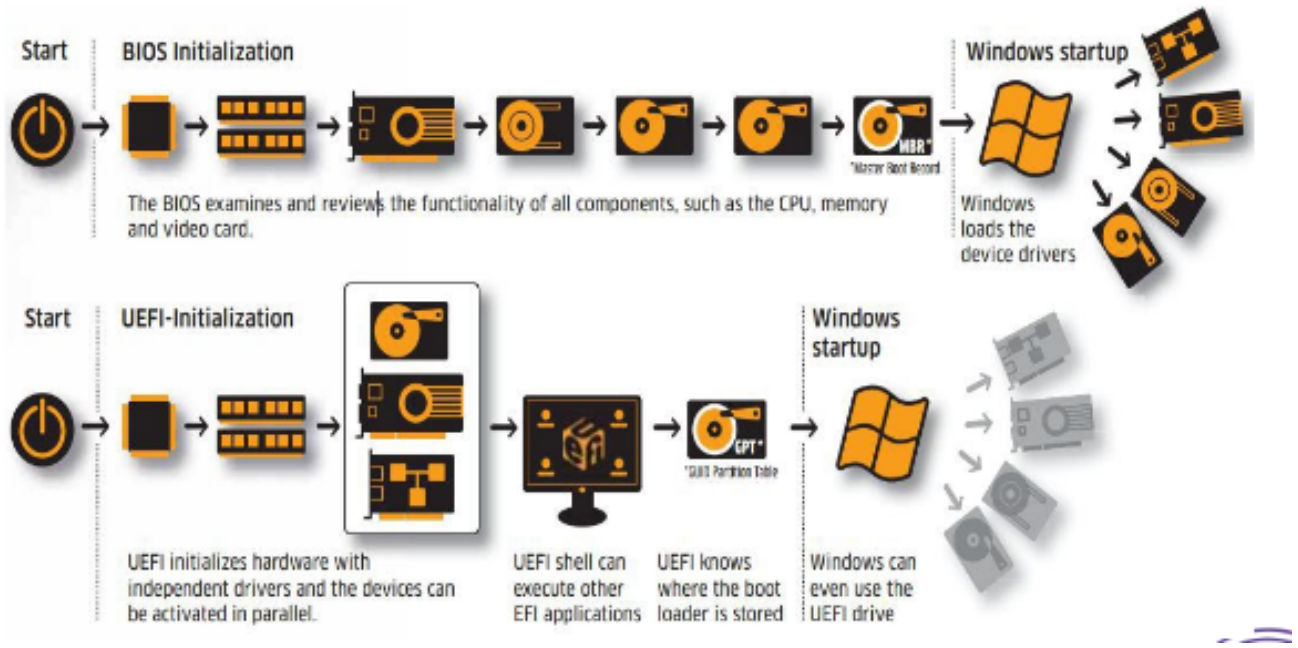
- **Interfaccia Grafica (GUI):** UEFI può offrire un'interfaccia grafica più intuitiva, con supporto al mouse, rendendo più semplice la navigazione e la configurazione delle impostazioni.
- **Boot Manager:** Un programma interno a UEFI che stabilisce quale sistema operativo o boot loader avviare. Questo semplifica il processo di avvio e offre più flessibilità rispetto al BIOS.
- **Supporto Driver:** UEFI può caricare driver direttamente, garantendo migliore compatibilità hardware e prestazioni già nella fase di avvio.
- **Sicurezza Avanzata:** UEFI include funzionalità come il *Secure Boot*, che protegge il sistema caricando solo software affidabile.
- **Estendibilità e Aggiornamenti:** UEFI è modulare e può essere aggiornato o esteso con nuove funzionalità, fornendo così supporto a nuove tecnologie e maggiore stabilità nel tempo.

GPT (GUID Partition Table)

La GUID Partition Table (GPT) è un moderno sistema per la gestione delle partizioni su un disco rigido o SSD.
Fa parte dello standard UEFI e sostituisce il vecchio **MBR** **(Master Boot Record)**.

- **GUID (Globally Unique Identifier):** ogni partizione ha un identificatore univoco a livello globale.
- **Struttura del GPT:**
  - **Primary GPT Header:** contiene la posizione e la struttura delle partizioni.
  - **Partition Entries:** ogni partizione ha un GUID e una descrizione.
  - **Secondary GPT Header (backup):** una copia di sicurezza dell'header primario, situata alla fine del disco.

Questa organizzazione offre maggiore **sicurezza** e **flessibilità** rispetto a MBR.



Bios VS UEFI.png

Drivers

I drivers sono programmi software che permettono al sistema operativo e ad altri software di comunicare con i dispositivi hardware.
Ad esempio, è grazie al **driver grafico** che un videogioco può comunicare correttamente con la scheda video e sfruttarne le potenzialità.

Il driver quindi **funge da intermediario:**

- **traduce le istruzioni ad alto livello provenienti dal sistema operativo in comandi a basso livello che l'hardware possa comprendere.**

Caratteristiche

- Sono installati sul sistema operativo.
- Facilitano la comunicazione tra software e hardware.
- Possono essere aggiornati o sostituiti frequentemente.

🔗 Parallelismo tra Driver e Bus

Possiamo pensare ai **driver** come a una sorta di "bus logico".

- I **bus hardware** servono a connettere e permettere la comunicazione tra i vari componenti della macchina (CPU, RAM, dispositivi di I/O, ecc.), trasferendo **segnali elettrici** e dati.
- I **driver**, invece, svolgono un ruolo simile ma sul piano software: fanno da **ponte tra il sistema operativo/applicazioni e i dispositivi hardware**,

traducendo i comandi software in istruzioni comprensibili dall'hardware.

Esempio pratico:

Quando un utente invia un documento in stampa, il sistema operativo genera un'istruzione software. Questa viene passata al **driver della stampante**, che la traduce in comandi a basso livello (impulsi e protocolli specifici) comprensibili dall'hardware della stampante stessa, consentendole di eseguire fisicamente la stampa.

Differenza tra firmware e driver

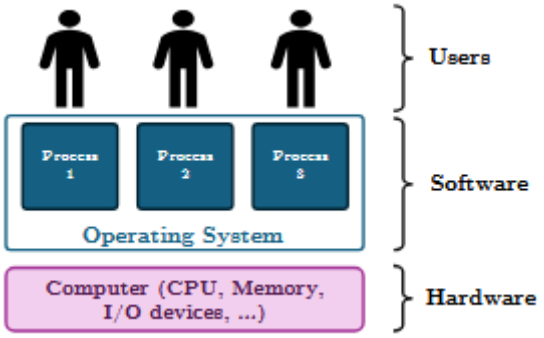
1. **La posizione:**
   - Al firmware non possiamo accedervi ne modificarlo mentre con il driver posso accedervi e modificarli
2. **Le funzionalità:**
   - il firmware ha il compito di avviare il componente mentre il driver è un ponte tra hardware e software
3. **La frequenza di aggiornamento:**
   - I drivers vengono aggiornati molto frequentemente, il firmware è aggiornato molto poco frequentemente e richiedono una procedura particolare.

Sistema Operativo

Il sistema operativo (SO) è il software più importante di un dispositivo, in quanto gestisce l'intera operatività della macchina.
Si colloca tra l'hardware (compreso il firmware) e il software applicativo, astrando la complessità del computer per gli utenti e i programmi.

Funzioni principali del sistema operativo

Il S.O. si occupa di **4 compiti fondamentali:**



Livelli SO.png

1. **Gestione dei componenti hardware:**
   - coordina CPU, memoria e dispositivi di I/O.
2. **Esecuzione di applicazioni, programmi e servizi:**
   - permette agli utenti di utilizzare il dispositivo.
3. **Interfacciamento con le periferiche:**
   - come tastiera, mouse, stampanti e dispositivi di archiviazione.
4. **Facilitazione dell'interazione con l'utente:**
   - offre un'interfaccia grafica o a riga di comando.

Ruolo del sistema operativo

Il sistema operativo definisce **2 aspetti chiave** di un dispositivo:

1. **Modalità operativa:**
   - determinando lo scopo e il funzionamento del dispositivo.
2. **Interazione con l'utente:**
   - specificando come l'utente può gestire e controllare la macchina.

Il SO è presente in tutti i dispositivi elettronici, inclusi computer, smartphone e sistemi embedded. In alcuni casi, il sistema operativo non prevede un'interfaccia diretta con l'utente, come accade nei dispositivi industriali o nei sistemi embedded.

Esempi di sistemi operativi

I principali sistemi operativi in uso includono:

- **Microsoft Windows**
- **GNU/Linux**
- **macOS**
- **Android**

- **iOS**
- **Altri sistemi specifici per ambienti embedded e industriali**

Classificazione dei Sistemi Operativi

I sistemi operativi possono essere classificati secondo due criteri principali: **caratteristiche di processing** e **caratteristiche funzionali**.

1. Classificazione dei sistemi operativi sulle caratteristiche di processing

I sistemi operativi possono essere classificati in base al modo in cui gestiscono l'elaborazione dei dati e l'interazione con l'utente:

1. **Batch Operating System (Sistema Batch):**
   - Non c'è interazione diretta con l'utente.
   - L'utente fornisce l'input, il sistema elabora i dati e restituisce l'output senza interventi intermedi(quindi solo a elaborazione completata).

> **Esempio**
>
> i primi mainframe IBM negli anni '60; oggi usati ancora in ambito bancario e nelle grandi elaborazioni di dati (es. elaborazione buste paga).

2. **Interactive Time-Sharing Operating System (Sistema interattivo a time-sharing):**
   - Esegue più applicazioni contemporaneamente in runtime, condividendo le risorse hardware tra i processi attivi.
   - Quindi l'utente può interagire con i programmi in esecuzione, fornendo input e ricevendo output in tempo reale.

> **Esempio**
>
> UNIX, Linux, Windows, macOS.

3. **Real-Time Operating System - RTOS (Sistema operativo real-time):**
   - Devono garantire che l'esecuzione dei processi avvenga entro un intervallo di tempo rigorosamente predefinito, senza ritardi né degradi di

performance.
- Utilizzati in ambiti critici come robotica, telecomunicazioni e sistemi industriali.

> **Esempio**
>
> FreeRTOS, QNX (utilizzato nei sistemi automotive), VxWorks, sistemi avionici e robotici.

4. **Embedded Operating System (Sistema Embedded):**
   - Integrati direttamente nell'hardware con uno scopo specifico
   - spesso senza interfaccia grafica e devono essere estremamente leggeri e ottimizzati.

> **Esempio**
>
> sistemi operativi dei microcontrollori per automobili, Android Automotive, sistemi avionici, firmware delle smart TV

5. **Hypervisor Operating System**:
   - Consentono la suddivisione delle risorse hardware in più macchine virtuali, permettendo l'esecuzione di più sistemi operativi contemporaneamente.

> **Esempio**
>
> VMware ESXi, Microsoft Hyper-V, Xen, KVM

2. Classificazione basata sulle caratteristiche funzionali

Inoltre i sistemi operativi vengono anche classificati in base alle loro funzionalità, ovvero quante risorse/utenti/processi il sistema riesce a gestire:

1. **Mono-Task Operating Systems(Monotasking):**
   - **Consentono l'esecuzione di un solo programma alla volta: il nuovo programma parte solo quando il precedente è terminato**.

> **Esempio**

MS-DOS

2. **Multi-Task Operating Systems:**
   - **Consentono l'esecuzione simultanea di più programmi, condividendo risorse come CPU e memoria**.

Esempio

Windows, Linux, MacOS.

3. **Multi-Threading Operating Systems:**
   - **Permettono la suddivisione di un singolo programma in più sotto-processi (thread) che possono essere eseguiti in parallelo, migliorando le prestazioni.**

Esempio

Java Virtual Machine su Windows/Linux, Android, Windows, sistemi Linux moderni.

4. **Mono-User Operating System(Monoutente):**
   - **Progettati per un solo utente alla volta, che ha accesso esclusivo alle risorse del sistema.**

Esempio

MS-DOS, vecchie versioni di Windows (Windows 95/98/ME).

5. **Multi-User Operating System:**
   - **Consentono l'accesso e l'esecuzione simultanea di programmi da parte di più utenti.**
   - **Ogni utente può avere sessioni indipendenti e risorse allocate in modo sicuro.**

Esempio

UNIX, Linux, Windows Server.

🖊 Nota:

Sistemi come Windows 10 o Linux Desktop sono tecnicamente multi-user, ma spesso vengono usati in modalità mono-user.

🔗 Relazione tra le due classificazioni

Le due classificazioni non sono alternative, ma si combinano tra loro:

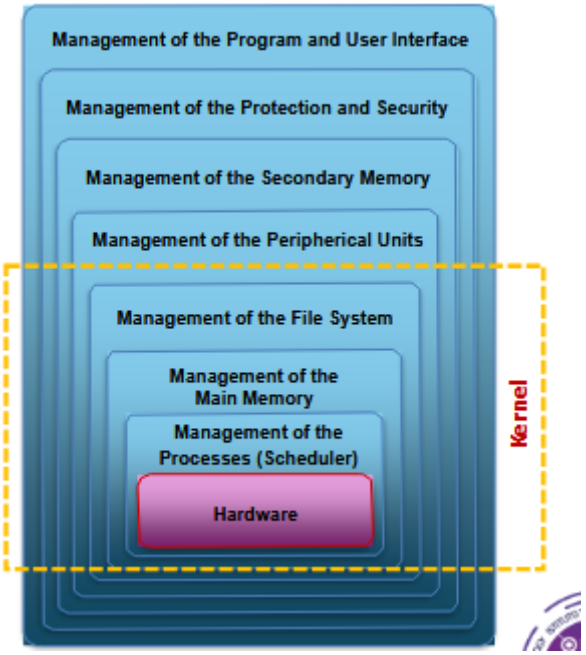
- Un **sistema multi-task** (funzionale) può essere anche un **time-sharing OS** (processing), ad esempio **Linux** o **Windows Server**, che gestiscono più processi e più utenti contemporaneamente.
- Un **sistema real-time (RTOS)** può essere sia **mono-task** (es. controllo semplice di un braccio robotico) sia **multi-threaded** (es. un robot industriale con più sensori e attuatori).
- Un **Hypervisor** permette di avere più sistemi operativi (ognuno multi-task e multi-user) che girano contemporaneamente sulla stessa macchina fisica.
- Un **sistema embedded** può essere **mono-user** **e** **mono-task** (es. firmware di un forno a microonde) oppure **multi-task** (es. centralina elettronica di un'auto moderna che gestisce più processi in parallelo).
- Un **sistema batch** invece è sempre **mono-user** e **mono-task** per natura, dato che non permette interazioni durante l'esecuzione.

✓ In sintesi:

- **La classificazione *processing* descrive:**
  - **come il sistema gestisce l'elaborazione.**
    (batch, time-sharing, real-time, embedded, hypervisor).
- **La classificazione *funzionale* descrive:**
  - **quante risorse/utenti/processi il sistema riesce a gestire.**
    (mono/multi-task, mono/multi-threading, mono/multi-user).

Struttura del SO

Un **Sistema Operativo (SO)** è organizzato in livelli di gestione, che vanno dall'hardware fino all'interfaccia con l'utente.
Ogni livello aggiunge funzionalità e astrazioni rispetto a quello sottostante.



Struttura dell'SO.png

Guardando l'immagine possiamo notare come ci siano almeno 8 livelli (compreso l'hardware), partiamo dal livello più alto andando verso il basso:

1. **Management of Programs and User Interface (Gestione dei Programmi e Interfaccia Utente:**
   - **È il livello più alto.**
   - **Fornisce all'utente i comandi e le interfacce grafiche (GUI o CLI) per interagire con il computer**.

> **Esempio:**
>
> Desktop di Windows, terminale Linux, icone, finestre, menu.

2. **Management of Protection and Security(Gestione della Protezione e Sicurezza):**
   - **Controlla che solo utenti e processi autorizzati possano accedere a risorse specifiche.**
   - **Include:**
     - **password, permessi sui file, crittografia, firewall, antivirus**.

- **Obiettivo:**
  - **proteggere dati e integrità del sistema.**

3. **Management of Secondary Memory (Gestione della Memoria Secondaria):**
   - Riguarda l'uso di dispositivi come **hard disk, SSD, memorie di massa esterne**.
   - **Il SO si occupa di archiviazione a lungo termine e della gestione dello spazio libero, delle directory, e dell'accesso ai dati**.
4. **Management of Peripheral Units (Gestione delle Unità Periferiche):**
   - **Coordina la comunicazione tra CPU e dispositivi esterni:**
     - stampanti, mouse, tastiere, schede di rete, monitor.
   - È qui che entrano in gioco i **driver**:
     - **traducono le istruzioni software in segnali comprensibili dall'hardware.**
5. **Management of the File System (Gestione del File System):**
   - **Organizza i dati nella memoria secondaria in file e directory.**
   - Permette operazioni come:
     - **creare, leggere, scrivere, cancellare, spostare file**.
   - **Garantisce anche i permessi di accesso (chi può leggere, scrivere o eseguire un file)**.
6. Management of Main Memory (Gestione della Memoria Principale):
   - Controlla l'uso della **memoria volatile (RAM)**.
   - Si occupa di:
     - **Allocare memoria ai processi**.
     - **Liberarla quando non serve più**.
     - **Implementare tecniche come paginazione e segmentazione**.
7. **Management of Processes (Gestione dei processi o anche detto Scheduler):**
   - **È la parte che gestisce l'esecuzione dei programmi attivi (processi)**.
   - Lo **scheduler** decide:
     - **quale processo eseguire**
     - **quando eseguirlo**
     - **e per quanto tempo lasciarlo in esecuzione**
   - **In sostanza garantisce equità e performance nell'esecuzione dei processi.**

- Usa algoritmi come: **Round Robin, FIFO, Priorità**.

In sintesi:

Possiamo vedere la struttura come una **piramide rovesciata**:

- Alla base c'è **l'hardware**.
- Subito sopra il **kernel**, che fa da "ponte".
- Poi i vari livelli di gestione.
- Infine, al vertice, l'**utente** che interagisce con tutto il sistema tramite l'interfaccia.

Kernel

Se guardiamo nel riguardo giallo dell'immagine notiamo la dicitura **Kernel:** **è la parte centrale e più vicina all'hardware del sistema operativo, fa da "ponte" tra l'hardware della macchina e il software (le applicazioni)**.

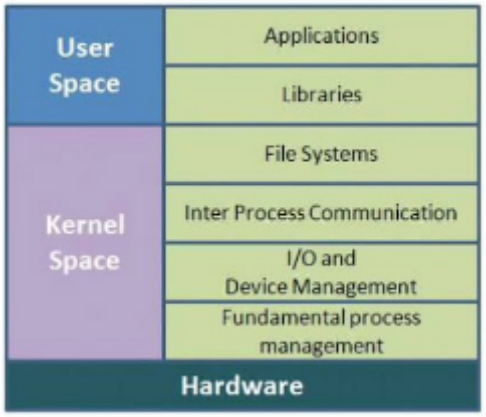
Quindi possiamo considerarlo come il cuore del SO.
Si occupa delle funzioni critiche come:

- **gestione processi**
- **gestione memoria**
- **gestione file system**
- **gestione periferiche**

Categorie di Kernel

Basandoci sulla loro struttura interna e sul design del kernel, i sistemi operativi possono essere categorizzati in 3 tipi differenti:

1. Monolithic Kernel Operating System :



Monolithic.png

- **Implementa una completa astrazione del computer all'interno del kernel, che gestisce direttamente il sistema operativo.**
- **Tutte le funzionalità essenziali (device driver, gestione memoria, gestione file system, system call) stanno nel Kernel Space, in un unico spazio di indirizzi.**
- **Le applicazioni e librerie girano invece nello User Space.**

Spiegazione dell'immagine

Guardando l'immagine possiamo notare due aree principali:

1. User Space (in blu):
Qui troviamo **Applicazioni** e **Librerie** che usano le funzionalità fornite dal kernel.

> **Esempio**
>
> Un programma di videoscrittura, che per salvare un file, si appoggia al file system che sta nel kernel.

2. Kernel Space (in viola-verde):

- Qui risiede tutto il "motore" del sistema operativo.
- Non c'è separazione modulare forte: **driver, file system, gestione dei processi, gestione I/O, comunicazione tra processi, tutto insieme nello stesso spazio**.

3. Hardware (in basso):

- Il kernel interagisce direttamente con l'hardware e fa da ponte tra questo e i programmi utente.

✓ Vantaggi

- **Prestazioni elevate** → perché c'è meno passaggio tra user mode e kernel mode (pochi context switch).
- **Maggiore efficienza** → i servizi comunicano tra loro direttamente all'interno del kernel.

⚠ Svantaggi

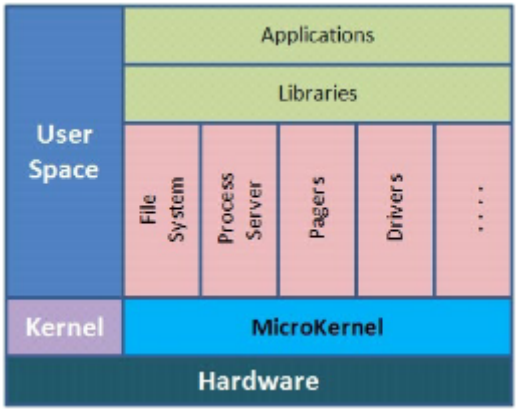
- **Stabilità ridotta** → un errore in un driver o in una parte del kernel può mandare in crash tutto il sistema.
- **Scarsa modularità** → aggiungere nuove funzionalità richiede modificare e ricompilare l'intero kernel.

🧠 In sintesi

Il **kernel monolitico** è come un unico "blocco" che contiene dentro sé tutti i servizi fondamentali del sistema operativo.
Mentre lo **user space:**
contiene solo i programmi che utilizzano questi servizi tramite le **system calls**.

2. Micro-kernel Operating System:

Un sistema operativo con micro-kernel implementa nel kernel solo le funzioni essenziali.
Tutte le altre funzionalità (driver, file system, server di processo, ecc.) vengono eseguite come processi separati nello spazio utente.



Microkernel.png

- **Kernel Space (spazio del kernel)**: contiene soltanto il “nucleo minimo”, cioè:
  - **Comunicazione tra processi (IPC)**
  - **Scheduling di base (gestione della CPU)**
  - **Astrazione minima dell’hardware**
- **User Space (spazio utente)**:
  qui troviamo i servizi che nel kernel monolitico erano integrati direttamente nel kernel:
  - **File system**
  - **Process server** (gestione avanzata dei processi)
  - **Driver dei dispositivi**
  - **Pagers** (gestione della memoria virtuale)
  - Altri server di sistema

Questi servizi comunicano tra loro e con il microkernel tramite **messaggi** (IPC).

📋 Spiegazione dell'immagine

1. **User Space (in rosa e verde)**
   - In alto: **Applicazioni** e **Librerie**, come prima.
   - Subito sotto: tanti “blocchi” separati (File System, Process Server, Drivers, ecc.).
   - Questi sono **moduli indipendenti** che girano nello spazio utente.
2. **Kernel Space (in blu)**
   - Qui troviamo solo il **MicroKernel**, cioè il nucleo minimo che gestisce comunicazione e scheduling.
   - Tutto il resto è stato spostato nello spazio utente.
3. **Hardware (in basso)**

- L'hardware continua a essere gestito, ma i driver non stanno più dentro al kernel: sono processi nello user space che comunicano con il microkernel.

✓ **Vantaggi**

- **Maggiore stabilità** → **un bug in un driver o in un servizio non manda in crash l'intero sistema, perché il kernel è isolato.**
- **Maggiore sicurezza** → **i servizi hanno meno privilegi e girano nello spazio utente.**
- **Modularità** → **è più facile aggiornare o sostituire parti del sistema (es. un driver)**

⚠ **Svantaggi**

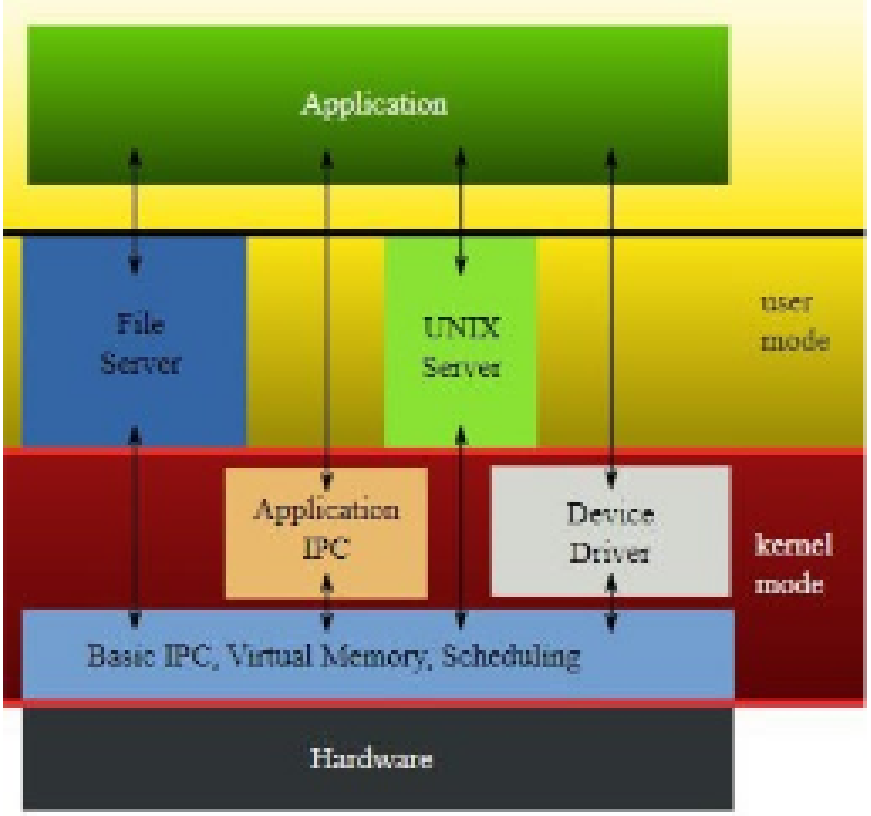
- **Prestazioni inferiori** rispetto al kernel monolitico → **perché i servizi devono continuamente comunicare tramite messaggi col microkernel, con più context switch.**
- **Maggiore complessità di progettazione** → **serve un'architettura più sofisticata per la comunicazione.**

🧠 **In sintesi:**

- Nel **monolitico** tutto è dentro al kernel (più veloce, ma meno stabile).
- Nel **microkernel** solo il minimo indispensabile è nel kernel, mentre i servizi girano nello user space (più sicuro e stabile, ma meno efficiente).

3. Hybrid kernel Operating System:

Un **kernel ibrido** è un'architettura che combina caratteristiche sia del kernel monolitico che del micro-kernel.



Hybrid Kernel.png

Struttura

Come mostrato nello schema:

- **User Mode**
  - Qui girano le **applicazioni** e alcuni **server** (es. File Server, UNIX Server).
  - I servizi sono più isolati e meno critici: se un server in user mode va in crash, non blocca l'intero sistema.
- **Kernel Mode**
  - Contiene i componenti essenziali per le prestazioni:
    - **Gestione della memoria virtuale**
    - **Scheduling dei processi**
    - **IPC di base (Inter-Process Communication)**
    - **Driver dei dispositivi principali**
  - Altri moduli (es. Application IPC, alcuni driver, filesystem aggiuntivi) vengono caricati dinamicamente per estendere le funzionalità del kernel.
- **Hardware**
  - Strato più basso, gestito dai driver attraverso il kernel.

Obiettivi

Il kernel ibrido cerca di bilanciare i vantaggi delle due architetture principali:

- **Dal kernel monolitico** eredita le **prestazioni**, perché i servizi critici rimangono nello spazio kernel e comunicano velocemente.
- **Dal microkernel** prende l'idea di una maggiore **modularità e stabilità**, grazie al fatto che diversi servizi possono essere implementati come moduli caricabili o processi in user mode.

Caratteristiche chiave

- **Performance**: i servizi vitali (memoria, CPU, driver critici) restano nel kernel.
- **Modularità**: moduli caricabili permettono di estendere le funzionalità senza ricompilare l'intero kernel.
- **Flessibilità**: parte dei servizi può essere spostata nello user mode per migliorare la stabilità.
- **Limite**: essendo i moduli comunque eseguiti in kernel space, un loro bug può compromettere la stabilità del sistema (non è sicuro quanto un puro microkernel).

Esempi pratici:

- **Microsoft Windows:** Il kernel di Windows (NT kernel) è un classico esempio di architettura ibrida:
  - Il file `ntoskrnl.exe` contiene il nucleo centrale,
  - mentre una miriade di file `.sys` (driver e moduli) vengono caricati per estenderne le funzionalità.
- **macOS (e iOS/iPadOS):** Il kernel XNU (X is Not Unix) di Apple è un ibrido che combina il nucleo del kernel Mach (un microkernel) con componenti del kernel BSD (che è monolitico) e un'interfaccia per i driver caricabili (I/O Kit).

In sintesi:

Un **kernel ibrido** è un kernel monolitico reso modulare. Mantiene nel kernel i componenti più importanti per la velocità, ma consente anche di caricare moduli e spostare parte dei servizi in user mode, ottenendo un compromesso tra **efficienza** e **flessibilità**.